Chromium Browser Artifact Parser

Programmer's Manual

Version: 2025-May-20 (Enhanced UI Edition)
Author: Jacques Boucher
Contact: jjrboucher@gmail.com
Repository: https://github.com/jjrboucher/Chromium-Browser-Artifact-Parser

---

Table of Contents

1. Architecture Overview
2. Project Structure
3. Core Components
4. Module Reference
5. Data Flow
6. SQLite Query Reference
7. Extending the Application
8. Chromium Database Schemas
9. Timestamp Handling
10. Contributing Guidelines

---

1. Architecture Overview

Design Philosophy

The Chromium Browser Artifact Parser follows a modular architecture that separates concerns into distinct components:

- **GUI Layer** - Tkinter-based interface handling user interaction
- **Parser Modules** - Individual modules for each artifact type
- **Data Processing** - Pandas-based data manipulation and transformation
- **Output Generation** - Excel file creation using openpyxl

Technology Stack

Component	Technology	Purpose
GUI Framework	Tkinter / ttk	Cross-platform user interface
Data Processing	Pandas	DataFrame operations and data manipulation
Database Access	sqlite3	SQLite database queries
Excel Output	openpyxl	Excel file creation and manipulation
JSON Parsing	json (stdlib)	Preferences and Bookmarks parsing

Threading Model

The application uses Python's `threading` module to prevent UI freezing during processing:

```python
self.processing_thread = threading.Thread(target=self.run_parser)
self.processing_thread.daemon = True
self.processing_thread.start()
```

The main thread handles UI updates while the worker thread processes artifacts.

---

2. Project Structure

```
Chromium-Browser-Artifact-Parser/
├── browser-artifact-parser-GUI.py    # Main application entry point
├── browser-artifact-parser-GUI.exe   # Windows executable (optional)
├── requirements.txt                  # Python dependencies
├── README.md                         # Basic documentation
│
├── Classes/
│   └── Preferences.py                # Preferences file parser class
│
├── Functions/
│   └── write_to_excel.py             # Excel output utility
│
├── JSON/
│   ├── bookmarks.py                  # Bookmarks JSON parser
```

```
│   └── bookmarks_hashing.py        # Bookmark hash utilities
│
├── SQLite/
│   ├── history.py                  # History database queries
│   ├── downloads.py                # Downloads queries
│   ├── cookies.py                  # Cookies queries
│   ├── favicons.py                 # Favicons queries
│   ├── logindata.py                # Login Data queries
│   ├── searchterms.py              # Search terms queries
│   ├── shortcuts.py                # Shortcuts queries
│   ├── topsites.py                 # Top Sites queries
│   ├── WebData.py                  # Web Data queries (autofill, etc.)
│   └── webasssist.py               # Edge-specific WebAssist queries
│
└── images/
    └── browser_chromium_icon.png   # Application icon
```

---

3. Core Components

3.1 Main Application Class

File: `browser-artifact-parser-GUI.py`
Class: `ModernChromeParserGUI`

The main class orchestrates the entire application:

```python
class ModernChromeParserGUI:
    def __init__(self, root):
        self.root = root
        self.profile_path = None
        self.output_path = None
        self.is_processing = False
        self.artifact_vars = {}

        self.setup_styles()
        self.setup_artifact_selection()
        self.create_widgets()
```

Key Methods:

Method	Purpose
`setup_styles()`	Configure ttk widget styles
`setup_artifact_selection()`	Initialize artifact checkboxes
`create_widgets()`	Build the GUI layout
`run_parser_threaded()`	Start processing in background thread
`run_parser()`	Main processing logic
`get_dataframes()`	Execute SQL and return DataFrame
`process_search_terms()`	Special handling for search terms
`process_bookmarks()`	Parse JSON bookmarks
`process_preferences()`	Parse Preferences file
`reorganize_workbook()`	Reorder Excel worksheets

3.2 Artifact Configuration

Artifacts are defined in a configuration dictionary:

```python
self.artifacts_config = {
    'History': {'enabled': True, 'query': 'History'},
    'History Gaps': {'enabled': True, 'query': 'History Gaps'},
    'Downloads': {'enabled': True, 'query': 'Downloads'},
    # ... additional artifacts
    'Web Assist (Edge)': {'enabled': False, 'query': 'Web Assist'}
}
```

Each artifact has:

- `enabled` - Default checkbox state
- `query` - Internal identifier for processing logic

3.3 Query Mappings

The `run_parser()` method maps artifacts to their database files and query functions:

```python
chromium_queries = {
    'History': [f'{self.profile_path}/History', chrome_history],
    'History Gaps': [f'{self.profile_path}/History', chrome_history_gaps],
    'Downloads': [f'{self.profile_path}/History', chrome_downloads],
    'Autofill': [f'{self.profile_path}/Web Data', chrome_autofill],
    'Cookies': [f'{self.profile_path}/Network/Cookies', chrome_cookies],
    # ... additional mappings
}
```

---

4. Module Reference

4.1 Classes/Preferences.py

Purpose: Parse the browser's Preferences JSON file to extract user and configuration data.

Class: `Preferences`

```python
class Preferences:
    def __init__(self, pref_file):
        self.pref_file = pref_file
        with open(self.pref_file, 'r', encoding='utf-8') as p:
            self.prefs = json.load(p)
```

Public Methods:

Method	Returns	Description
email()	str	Google account email
full_name()	str	User's full name
gaia()	str	Google Account ID (GAIA)
given_name()	str	User's first name
profile_name()	str	Chrome profile name
edge_first_name()	str	Edge-specific first name
edge_last_name()	str	Edge-specific last name
thumbnail_url()	str	Profile picture URL
language()	str	Account locale
privacy_settings()	dict	Browser clear data settings
country_id()	str	Country code at install
profile_created_date()	str	Profile creation timestamp
profile_created_version()	str	Browser version at creation
download_directory()	str	Default download path
save_file_directory()	str	Last "Save As" directory
prompt_for_download()	bool	Whether to prompt for download location
new_tab()	str	Custom new tab shortcuts
startup()	str	Startup behavior setting
startup_urls()	str	URLs opened at startup
homepage()	str	Homepage URL
homepagenewtab()	bool	Is homepage a new tab
exit_type()	str	Last exit status (Normal/Crashed)
previousNavigationTime()	str	Last navigation timestamp

String Representation:

The `__str__` method provides formatted output of all parsed preferences, used when writing to the Excel file.

4.2 Functions/write_to_excel.py

Purpose: Utility function to write DataFrames to Excel files.

```python
def write_excel(dataframe, worksheet_name, excel_file):
    """
    Write the dataframe to an Excel file

    Args:
        dataframe: pandas DataFrame to write
        worksheet_name: Name for the worksheet
        excel_file: Output file path
    """
    if os.path.isfile(excel_file):
        # Append to existing file
        with pd.ExcelWriter(excel_file, engine='openpyxl', mode='a') as writer:
            dataframe.to_excel(writer, sheet_name=worksheet_name, index=False)
    else:
        # Create new file
        with pd.ExcelWriter(excel_file, engine='openpyxl', mode='w') as writer:
            dataframe.to_excel(writer, sheet_name=worksheet_name, index=False)
```

4.3 JSON/bookmarks.py

Purpose: Parse Bookmarks and Bookmarks.bak JSON files.

Key Functions:

```python
```

```python
def convert_webkit_timestamp(webkit_timestamp):
    """Convert WebKit timestamp to datetime object"""
    base_date = datetime(1601, 1, 1)
    timestamp_in_seconds = webkit_timestamp / 1_000_000
    return base_date + timedelta(seconds=timestamp_in_seconds)

def get_chromium_bookmarks(bookmark_path):
    """
    Parse bookmarks file and return DataFrame

    Args:
        bookmark_path: Path to Bookmarks or Bookmarks.bak

    Returns:
        tuple: (DataFrame, worksheet_name)
    """
```

Output Columns:

- Source, Type, Folder Path, ID, Name
- Date added (raw and converted)
- Date last used (raw and converted)
- Date Modified (raw and converted, folders only)
- URL

4.4 SQLite Query Modules

Each SQLite module follows a consistent pattern:

```python
def chrome_artifact_name():
    worksheet = "Worksheet Name"
    sql_query = """
        SELECT column1, column2,
            datetime(timestamp/1000000-11644473600,'unixepoch') AS 'Decoded timestamp (UTC)'
        FROM table_name
        JOIN other_table ON ...
        ORDER BY ...
    """
    return sql_query, worksheet
```

Module Summary:

Module	Functions	Database File
history.py	chrome_history(), chrome_history_gaps()	History
downloads.py	chrome_downloads(), chrome_downloads_gaps()	History
cookies.py	chrome_cookies()	Network/Cookies
favicons.py	chrome_favicons()	Favicons
logindata.py	chrome_login_data(), chrome_login_data_gaps()	Login Data
searchterms.py	chrome_keyword_historyquery()	History
shortcuts.py	chrome_shortcuts()	Shortcuts
topsites.py	chrome_topsites()	Top Sites
WebData.py	chrome_autofill(), chrome_keywords(), chrome_masked_credit_cards(), chrome_masked_bank_accounts(), chrome_addresses(), chrome_loyalty_cards()	Web Data
webasssist.py	edge_webassist()	WebAssistDatabase

---

5. Data Flow

Processing Pipeline

```
┌──────────────────┐
│  User Input      │
│ (Profile Path)   │
└──────────┬───────┘
           │
           ▼
┌──────────────────┐
│ Artifact         │
│ Selection        │
└──────────┬───────┘
           │
           ▼
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│          For Each Selected Artifact          │
├──────────────────────────────────────────────────────────────────────────┤
│                                │
│  ┌──────────────────────┐   ┌──────────────────────┐        │
│  │ SQLite Query │ OR │ JSON Parser  │        │
│  │   Module     │   │   Module     │        │
│  └──────────┬───────────┘   └──────────┬───────────┘        │
│       │            │            │
│       └──────────┬─────────────┘            │
│           ▼                  │
│       ┌──────────────────────┐             │
│       │   Pandas     │              │
│       │  DataFrame   │              │
│       └──────────┬───────────┘             │
│           │                  │
│           ▼                  │
│       ┌──────────────────────┐             │
│       │ write_excel() │              │
│       └──────────────────────┘             │
│                                │
└──────────────────────────────────────────────────────────────────────────┘
      │
      ▼
┌──────────────────────┐
│ Summary Sheet  │
│  Generation    │
└──────────┬───────────┘
      │
      ▼
┌──────────────────────┐
│  Workbook      │
│ Reorganization │
└──────────┬───────────┘
      │
      ▼
┌──────────────────────┐
│ Output .xlsx   │
└──────────────────────┘
```

DataFrame Processing

The `get_dataframes()` method handles SQLite-based artifacts:

```python
def get_dataframes(self, db_file, function):
    """Get dataframes from SQLite database"""
    query, worksheet_name = function()  # Get SQL and worksheet name
    conn = sqlite3.connect(db_file)     # Connect to database
    dataframe = pd.read_sql_query(query, conn)  # Execute query
    conn.close()
    return dataframe, worksheet_name
```

Special Processing: Search Terms

Search terms require merging data from two sources:

```python
def process_search_terms(self):
    # Get keyword_search_terms from History
    df_history, _ = self.get_dataframes(
        f'{self.profile_path}/History',
        chrome_keyword_historyquery
    )

    # Get keywords from Web Data
    df_keywords, _ = self.get_dataframes(
        f'{self.profile_path}/Web Data',
        chrome_keywords
    )

    # Merge based on keyword_id
    searchterms = []
    for row in df_history.itertuples():
        if not np.isnan(row[3]):  # keyword_id exists
            kw = df_keywords.query(f'id == {row[3]}')
            # ... build merged record

    return pd.DataFrame(searchterms), 'Search Terms'
```

---

6. SQLite Query Reference

Timestamp Conversion Formula

Chromium uses WebKit timestamps (microseconds since January 1, 1601):

```sql
datetime(timestamp/1000000-11644473600,'unixepoch') AS 'Decoded timestamp (UTC)'
```

Breakdown:

- `/1000000` - Convert microseconds to seconds
- `-11644473600` - Offset from 1601 to Unix epoch (1970)
- `'unixepoch'` - Tell SQLite to interpret as Unix timestamp

Transition Type Decoding

The `transition` field is a 32-bit integer with multiple flags:

```sql
-- Core transition type (rightmost byte)
CASE (transition&0xff)
   WHEN 0 THEN 'Clicked on a link'
   WHEN 1 THEN 'Typed URL'
   WHEN 2 THEN 'Clicked on suggestion'
   -- ... etc
END AS 'Transition Type'

-- Qualifier flags (bits 24-32)
CASE (transition&0x01000000)
   WHEN 0x01000000 THEN 'yes'
END AS 'Forward/Back button'

CASE (transition&0x02000000)
   WHEN 0x02000000 THEN 'yes'
END AS 'From Address Bar'
```

Gap Detection Pattern

Used in `history_gaps`, `downloads_gaps`, and `logindata_gaps`:

```sql
```

```sql
-- Find gaps using LAG window function
SELECT * FROM (
  SELECT
    LAG(ROWID,1) OVER (ORDER BY ROWID) AS "Previous",
    ROWID,
    (ROWID - LAG(ROWID,1) OVER (ORDER BY ROWID) - 1) AS "Missing",
    LAG(datetime(...),1) OVER (ORDER BY ROWID) AS "Start Time",
    datetime(...) AS "End Time"
  FROM table_name
)
WHERE ROWID - "Previous" > 1

-- Check for missing records at the end
UNION
SELECT ...
FROM table_name
WHERE id = (SELECT MAX(id) FROM table_name)
 AND id < (SELECT seq FROM sqlite_sequence WHERE name='table_name')
```

Complex JOIN Examples

Favicons (3-table join):

```sql
SELECT
  icon_mapping.page_url,
  favicons.url AS 'favicon URL',
  favicon_bitmaps.image_data,
  (favicon_bitmaps.height || " X " || favicon_bitmaps.width) AS "dimensions"
FROM icon_mapping
  LEFT JOIN favicons ON favicons.id == icon_mapping.icon_id
  LEFT JOIN favicon_bitmaps ON favicon_bitmaps.icon_id == favicons.id
```

Addresses (multiple self-joins):

```sql
```

```sql
SELECT addresses.guid,
   atk_first_name.value AS "First Name",
   atk_last_name.value AS "Last Name",
   -- ... etc
FROM addresses
LEFT JOIN (SELECT * FROM address_type_tokens WHERE type == 3)
   AS atk_first_name ON addresses.guid == atk_first_name.guid
LEFT JOIN (SELECT * FROM address_type_tokens WHERE type == 5)
   AS atk_last_name ON addresses.guid == atk_last_name.guid
-- ... additional joins for each field type
```

Bank Accounts (with metadata join):

```sql
SELECT  mba.instrument_id,
        mba.bank_name AS "Bank Name",
        mba.account_number_suffix AS "Account Number Suffix",
        mba.account_type AS "Account Type",
        mba.nickname,
        mbam.use_count,
        mbam.use_date,
        DATETIME(mbam.use_date/1000000-11644473600,'unixepoch') AS "Decoded use_date (UTC)"
FROM masked_bank_accounts AS mba
LEFT JOIN masked_bank_accounts_metadata AS mbam
   ON mba.instrument_id == mbam.instrument_id
```

Loyalty Cards (with merchant domain join):

```sql
SELECT  lc.merchant_name,
        lcmd.merchant_domain,
        lc.program_name,
        lc.program_logo,
        lc.loyalty_card_number
FROM loyalty_cards AS lc
LEFT JOIN loyalty_card_merchant_domain AS lcmd
   ON lc.loyalty_card_id == lcmd.loyalty_card_id
```

---

7. Extending the Application

Adding a New Artifact

Step 1: Create the query module in `SQLite/`:

```python
# SQLite/new_artifact.py

def chrome_new_artifact():
    worksheet = "New Artifact"
    sql_query = """
        SELECT column1, column2,
            datetime(timestamp/1000000-11644473600,'unixepoch')
                AS 'Decoded timestamp (UTC)'
        FROM new_table
        ORDER BY timestamp
    """
    return sql_query, worksheet
```

Step 2: Import in main script:

```python
from SQLite.new_artifact import chrome_new_artifact
```

Step 3: Add to artifact configuration:

```python
self.artifacts_config = {
    # ... existing artifacts
    'New Artifact': {'enabled': True, 'query': 'New Artifact'},
}
```

Step 4: Add to query mappings:

```python
chromium_queries = {
    # ... existing queries
    'New Artifact': [f'{self.profile_path}/Database', chrome_new_artifact],
}
```

Adding Browser-Specific Handling

For browser-specific artifacts (like Edge's WebAssist), add to the separate mapping:

```python
edge_queries = {
    'Web Assist (Edge)': [f'{self.profile_path}/WebAssistDatabase', edge_webassist],
    'New Edge Feature': [f'{self.profile_path}/EdgeDatabase', edge_new_feature],
}
```

Adding a New Preferences Field

In `Classes/Preferences.py`:

```python
def new_preference(self):
    try:
        value = self.prefs.get("parent_key").get("child_key")
    except (KeyError, IndexError, AttributeError, TypeError):
        value = "not found"
    return value

def __str__(self):
    # Add to the formatted output
    return (f'...\n'
            f'New Preference: {self.new_preference()}\n'
            f'...')
```

Creating a Custom Output Format

To add output formats beyond Excel:

```python
# Functions/write_to_csv.py

def write_csv(dataframe, filename, output_dir):
    """Write DataFrame to CSV file"""
    filepath = os.path.join(output_dir, f'{filename}.csv')
    dataframe.to_csv(filepath, index=False)
    return filepath
```

---

8. Chromium Database Schemas

History Database

Tables:

- `urls` - URL records
- `visits` - Visit records (linked to urls)
- `visit_source` - Origin of visit (local/synced)
- `segments` - URL path segments
- `keyword_search_terms` - Search queries

Key Relationships:

```
urls.id ←── visits.url
visits.id ←── visit_source.id
visits.segment_id ──→ segments.id
urls.id ←── keyword_search_terms.url_id
```

Web Data Database

Tables:

- `autofill` - Form field entries
- `addresses` - Saved addresses
- `address_type_tokens` - Address field types
- `masked_credit_cards` - Credit card info
- `server_card_metadata` - Credit card usage stats
- `masked_bank_accounts` - Bank account info
- `masked_bank_accounts_metadata` - Bank account usage stats
- `loyalty_cards` - Loyalty/rewards card info
- `loyalty_card_merchant_domain` - Loyalty card merchant domains
- `keywords` - Search engine definitions

Login Data Database

Tables:

- `logins` - Saved credentials
- `stats` - Usage statistics (newer versions)

Favicons Database

Tables:

- `icon_mapping` - URL to favicon mapping
- `favicons` - Favicon metadata
- `favicon_bitmaps` - Favicon image data

Cookies Database (Network/Cookies)

Tables:

- `cookies` - Cookie records with timestamps

Top Sites Database

Tables:

- `top_sites` - Most visited sites

Shortcuts Database

Tables:

- `omni_box_shortcuts` - Address bar shortcuts

---

9. Timestamp Handling

WebKit Timestamp Format

Chromium uses WebKit timestamps:

- **Unit:** Microseconds
- **Epoch:** January 1, 1601 00:00:00 UTC
- **Range:** 64-bit integer

Conversion Methods

SQLite (in-query):

```sql
```

```
datetime(timestamp/1000000-11644473600,'unixepoch')
```

Python:

```python
from datetime import datetime, timedelta

def convert_webkit_timestamp(webkit_timestamp):
    base_date = datetime(1601, 1, 1)
    timestamp_in_seconds = webkit_timestamp / 1_000_000
    return base_date + timedelta(seconds=timestamp_in_seconds)
```

Handling Special Values

Some timestamp fields have special meanings:

```sql
-- Zero typically means "never" or "not applicable"
CASE timestamp
    WHEN 0 THEN 'Never'
    ELSE datetime(timestamp/1000000-11644473600,'unixepoch')
END
```

Unix Timestamps

Some fields (like autofill dates) use Unix timestamps:

```sql
datetime(date_created,'unixepoch') AS 'Decoded date_created (UTC)'
```

---

10. Contributing Guidelines

Code Style

- Follow PEP 8 Python style guidelines
- Use descriptive variable and function names
- Include docstrings for all functions and classes
- Comment complex SQL queries extensively

SQL Query Standards

- Include header comments with author, date, and tested versions
- Decode all timestamp fields with consistent column naming
- Handle NULL values appropriately
- Use CASE statements for enumerated values
- Order results by timestamp when applicable

Testing New Features

1. Test against multiple browser versions
2. Test with empty databases
3. Test with large datasets
4. Verify timestamp conversion accuracy
5. Check Excel output formatting

Submitting Changes

1. Fork the repository
2. Create a feature branch
3. Make changes with appropriate tests
4. Update documentation
5. Submit a pull request

Reporting Issues

Include:

- Browser type and version
- Operating system
- Error messages (full stack trace)
- Sample database files (if possible, redacted)

Contact

For questions, suggestions, or contributions:

- **Email:** jjrboucher@gmail.com
- **GitHub Issues:** https://github.com/jjrboucher/Chromium-Browser-Artifact-Parser/issues

Appendix A: Chromium Source Code References

Transition Types:

- https://cs.chromium.org/chromium/src/ui/base/page_transition_types.h

History Types:

- https://cs.chromium.org/chromium/src/components/history/core/browser/history_types.h

Download Danger Types:

- https://source.chromium.org/chromium/chromium/src/+/main:components/download/public/common/download_danger_type.h

Download Interrupt Reasons:

- https://source.chromium.org/chromium/chromium/src/+/main:components/download/public/common/download_interrupt_reason_values.h

Favicon Types:

- https://source.chromium.org/chromium/chromium/src/+/main:components/favicon/core/favicon_database.cc

Password Types:

- https://source.chromium.org/chromium/chromium/src/+/main:components/password_manager/core/browser/password_manager_metrics_util.h

Appendix B: Value Lookup Tables

Visit Source Values

Value	Meaning
0	Synced
1 (or NULL)	Local
2	Extension
3	Firefox Imported

Value	Meaning
4	IE Imported
5	Safari Imported

Download Danger Types

Value	Meaning
0	Not Dangerous
1	Dangerous File
2	Dangerous URL
3	Dangerous Content
4	Maybe Dangerous
5	Uncommon Content
6	User Validated
7	Dangerous Host
8	Potentially Unwanted
9-23	Various security states

Favicon Types

Value	Meaning
0	Invalid
1	Favicon
2	Touch Icon
3	Touch Precomposed Icon
4	Web Manifest Icon

Document Version: 1.0
Last Updated: January 2026